Student Name : Maneesha Eeshwara – 300392759

Project : FarmVendor - Smart Supply Chain Management Platform

Course : W26-4495

Reporting Period : 17-Mar-2026 to 23-Mar-2026

Progress Report #5

Work Date/Hours Log

Date	No of Hours	Description of work done
18-Mar	4	Implemented and tested the farmer–vendor chat module in the FarmVendor system. Added chat conversation creation and message flow between farmer and vendor users.
		Fixed login-related user identification issues by updating Login.jsx to store userId in localStorage correctly.
		Resolved chat database and foreign key issues by verifying valid farmer and vendor IDs from the AspNetUsers table and testing conversation creation successfully.
		Fixed SignalR/CORS connection issues by updating backend configuration in Program.cs and retesting real-time chat connection.
		Improved chat usability by replacing manual user ID entry with searchable selection of existing farmers and vendors using name/email search. Updated ChatController.cs, chatApi.js, FarmerChat.jsx, and VendorChat.jsx.
		Tested the full chat workflow from both farmer and vendor sides, including conversation opening, message display, and sending messages in a more user-friendly interface.
19-Mar	3	Enhanced user experience of forecasting interface by replacing manual userId inputs with searchable dropdowns for vendor and product selection in FarmerForecast.jsx. Developed backend APIs for vendor and product lookup with search functionality in DemandForecastController.cs (endpoints: /vendors, /products). Updated API integration in forecastApi.js to support dynamic search queries and data fetching. Improved validation handling and request payload structure in forecasting workflow to correctly include vendorId and prevent API errors.
		Updated the navigation bar by creating a separate common component file. Refactored FarmerSidebar.jsx, VendorSidebar.jsx, and App.jsx to import the shared component. All related .jsx files were updated to include the new import structure and integrate the sidebar elements.

20-Mar	3	Implemented ML-based vendor recommendation feature to identify top vendors for dispatching based on forecast demand, inventory availability, and historical request data. Developed backend logic in FarmerRecommendationService.cs and exposed API via FarmerController.cs (/recommended-vendors). Integrated frontend in FarmerDashboard.jsx to display top recommended vendors dynamically.
22-Mar	3	Populated the system with additional historical demand data to support forecasting experiments and verified that ML.NET SSA forecast rows were generated successfully in the DemandForecast table.
23-Mar	5	Debugged why recommended vendors and charts were not appearing by checking forecast dates, inventory overlap, API responses, and endpoint usage. Fixed forecasting chart integration by correcting the chart API endpoint in forecastApi.js, updating chart-loading logic to use the selected forecasting model, and refining DemandForecastService.cs chart data retrieval to return recent historical values together with matching forecast values from the selected date onward. Continued testing of farmer-specific recommended vendors and confirmed that recommendation results change based on the logged-in farmer's inventory and forecast matching data.
		Progress report 5 submission

Summary Description of Work Done During This Reporting Period

During this week, the FarmVendor system progress was made in enhancing both the functionality and user experience of the FarmVendor system through full-stack development and integration.

On March 18, a complete real-time farmer–vendor chat module was designed, implemented, and tested using SignalR within the ASP.NET Core and React architecture. This included enabling conversation creation and real-time message exchange between users. Several critical issues were resolved during this phase, including incorrect user identification in the login flow, foreign key constraint errors in the database, and SignalR connection issues related to CORS and backend configuration. The chat interface was further improved by replacing manual user ID entry with a searchable dropdown based on user name and email, making the system more intuitive and user-friendly. End-to-end

testing was successfully conducted from both farmer and vendor perspectives to ensure reliable communication.

On March 19, the forecasting module interface was enhanced to improve usability and data accuracy. Manual inputs for vendor and product selection were replaced with dynamic, searchable dropdowns, supported by newly developed backend APIs for vendor and product lookup. The frontend API integration was updated to handle dynamic queries, and validation logic was improved to ensure correct request payloads and prevent runtime errors. Additionally, the application navigation was refactored by introducing a shared sidebar component, reducing code duplication and improving maintainability across multiple pages.

On March 20, an advanced vendor recommendation feature was implemented to support intelligent dispatch planning. This feature leverages forecasting outputs, farmer inventory data, and historical demand patterns to identify and rank the most suitable vendors. The recommendation logic was developed in the backend service layer and exposed via a secured API endpoint. The frontend dashboard was enhanced to dynamically display the top recommended vendors, enabling better decision-making for farmers and transforming the system into a data-driven platform.

On March 22, the system was populated with sufficient historical demand, inventory, and dispatch data to support forecasting, recommendation, and optimization experiments. ML.NET SSA forecasting was executed successfully, and forecast records were generated and stored in the DemandForecast table, enabling further analytical and recommendation functionalities.

On March 23, extensive debugging and refinement were carried out to resolve issues related to missing chart data and vendor recommendations. Problems related to API endpoint mismatches, incorrect parameter passing, forecast date alignment, and model selection were identified and fixed. The chart integration was corrected by updating the forecast API endpoint, improving frontend chart-loading logic, and refining the backend DemandForecastService to properly filter and return both historical and forecast data. Additional validation and testing were performed to ensure that recommended vendors dynamically change based on the logged-in farmer’s inventory and forecast conditions.

Overall, the work during this period focused on real-time communication, intelligent forecasting integration, data-driven vendor recommendation, system usability improvements, and robust frontend–backend integration. These enhancements significantly improved system reliability, user experience, and decision support

capabilities, advancing the FarmVendor platform toward a scalable and intelligent supply chain solution.

Next Step

1. Generate analytical outputs, including graphs and tables, to visualize forecasting trends and recommendation effectiveness.
2. Evaluate system performance using metrics such as demand fulfillment rate, inventory utilization, and recommendation accuracy.
3. Analyze relationship score trends between farmers and vendors based on delivery outcomes.
4. Modify final User interface for more professional output
5. Capture screenshots and system outputs to support experimental findings.
6. Begin preparing the final report, including methodology, implementation details, experimental results, and discussion.
7. Prepare final presentation slides

Repo Check in of Implementation completed

The implementation work completed during this reporting period has been periodically checked into the GitHub repository under the Implementation folder. The files and folders I have checked into the repository are as follows:

- **Backend Implementation (files added/updated):**
    - DemandForecastController.cs
    - ChatController.cs
    - FarmerRecommendationService.cs
    - RecommendedVendorRowDto.cs
    - ChatUserLookupDto.cs
    - FarmerController.cs
    - Program.cs

- **Frontend Implementation (files added/updated):**
    - forecastApi.js
    - chatApi.js

    - FarmerSidebar.jsx
    - VendorSidebar.jsx
    - FarmerAnalytics.jsx
    - FarmerChat.jsx
    - FarmerDispatch.jsx
    - FarmerForecast.jsx
    - FarmerProducts.jsx
    - FarmerRequests.jsx
    - VendorAnalytics.jsx
    - VendorChat.jsx
    - VendorForecast.jsx
    - VendorIncoming.jsx
    - VendorRequests.jsx
    - VendorStock.jsx
    - App.jsx
    - Login.jsx
    - FarmerDashboard.jsx
    - VendorDashboard.jsx
    - package-lock.json

- **Database/sql queries:**
    - userID.sql

All changes have been committed to the repository under the appropriate Implementation directory. The progress report document will be checked into the DocumentsAndReports folder as required, and submitted on Blackboard.

- **DocumentsAndReports**
    - Progress reports5 document
    - Worklog excel sheet
    - Installation steps document
    - Issues identified document
    - Appendix
    - ML generation for top 5 vendors.docx
    - ManeeshaE_FinalReport.docx

AI Use Section

Date	Work Completed	Details / Description
18-Mar	Chat module integration and enhancement	Independently designed and integrated a real-time farmer–vendor chat system using SignalR within the existing ASP.NET Core + React architecture. Ensured secure communication using JWT-based authentication and proper user identification.
	Debugging and system stabilization	Identified and resolved complex issues related to foreign key constraints, incorrect userId handling, and SignalR CORS/connection errors through debugging and configuration fixes.
	User experience improvement	Enhanced usability by replacing manual ID-based chat initiation with a searchable dropdown using display name and email, making the system more practical and user-friendly.
	Full-stack integration	Successfully connected backend chat APIs, database persistence, and frontend UI components, ensuring end-to-end functionality including conversation creation, message flow, and real-time updates.
	System design improvement	Extended the system architecture to support scalable communication between farmers and vendors, aligning with the project goal of improving coordination and interaction within the supply chain platform.
19-Mar	Value added through UI/UX and system integration improvements	Improved overall system usability by eliminating manual ID entry and introducing intuitive search-based selection for vendors and products. Enabled scalable and user-friendly data selection using dynamic backend filtering, enhancing real-world applicability. Strengthened frontend–backend integration by aligning API contracts and validation logic, reducing runtime errors. Contributed to better data accuracy and user adoption by ensuring correct entity selection before forecasting operations.
	code complexity decresed by common navigationbar	Reduced React code complexity by implementing a common navigation bar and eliminated incorrect manual routing paths.
20-Mar	Vendor recommendation module implementation	Designed and implemented a data-driven vendor recommendation system using forecasting outputs and historical demand patterns.

		Combined ML.NET forecasting results with business logic to rank vendors based on demand, availability, and fulfillment potential.
		Enabled decision support for farmers by suggesting optimal vendors for dispatch planning.
		Extended system from a transactional platform into an intelligent recommendation-driven system.
22-Mar	Backend forecasting service refinement	Leveraged AI guidance to improve the DemandForecastService.cs chart data retrieval logic, ensuring proper filtering of historical demand and forecast data based on selected vendor, product, date, and model.
23-Mar	API integration and testing guidance	Used AI to validate correct API endpoint usage (/api/forecasts/chart) and parameter passing for vendorId, productId, forecastDate, and modelName. Assisted in debugging empty responses and aligning frontend–backend data flow.
	System validation and troubleshooting	Applied AI recommendations to verify data availability (historical demand and forecast records), test API responses using browser/Postman, and confirm that recommendation and chart features behave correctly based on selected inputs.
	Forecasting and chart debugging support	Used AI assistance to diagnose why forecasting charts were not displaying by identifying issues related to API endpoint mismatches, missing parameters, and incorrect model usage. Suggested corrections for frontend API calls and chart-loading logic.

Value Added Beyond AI

- Enabled seamless and secure farmer–vendor interaction through a fully functional real-time chat system, enhancing platform collaboration and responsiveness.

- Reduced system errors and failures by resolving authentication, database, and SignalR connection issues, ensuring smoother user experience and consistent performance.

- Replaced manual ID-based workflows with intuitive search-based selection, significantly improving ease of use and reducing user input errors.

- Achieved full integration between frontend, backend, and database layers, ensuring reliable data flow and real-time updates across the application.

- Designed a scalable communication module supporting multi-user interactions, aligning with real-world supply chain communication needs.

- Introduced dynamic search-based selection for vendors and products, improving usability, reducing invalid inputs, and enhancing overall data integrity.

- Implemented a reusable common navigation component, minimizing code duplication and simplifying future development and maintenance.

- Transformed the system into a data-driven platform by integrating ML-based vendor recommendations, enabling smarter dispatch decisions for farmers.

- Improved dispatch planning by ranking vendors based on forecast demand, availability, and historical performance, increasing fulfillment effectiveness.

- Leveraged AI-assisted debugging to quickly identify root causes of chart and API issues, significantly reducing troubleshooting time.

- Enhanced chart accuracy and relevance by ensuring proper filtering and alignment of historical and forecast data, leading to better analytical insights.

- Ensured correct API usage and parameter passing, reducing integration errors and improving overall system consistency.

- Validated data availability and API responses, ensuring forecasting, recommendation, and visualization features work correctly under real conditions.

Appendix – Prompt History

Below are representative prompts used during this reporting period:

1. "How to implement real-time chat functionality using SignalR in ASP.NET Core with React frontend"
2. "Fix foreign key constraint error when creating chat conversation between two users in EF Core"

3. “How to correctly store and retrieve authenticated userId from JWT/localStorage in React application”
4. “Resolve SignalR connection issues including CORS and WebSocket negotiation errors in ASP.NET Core”
5. “Design chat system with conversation creation, message storage, and real-time updates using SignalR”
6. “Replace manual user ID input with searchable dropdown using name/email in React UI”
7. “Create backend API to search users (vendors/farmers) by name or email using Entity Framework Core”
8. “Fix React module import/export errors and ensure proper API integration structure”
9. “Create API endpoints for vendor and product lookup with search functionality in ASP.NET Core”
10. “How to handle validation errors and payload structure issues in React form submission to backend APIs”
11. “Design recommendation system to suggest best vendors for farmers based on forecast and inventory data”
12. “Implement ML-based or rule-based vendor recommendation logic using forecast demand and available inventory”
13. “Create API endpoint to return top recommended vendors for a farmer dashboard”
14. “Integrate vendor recommendation API into React dashboard and display results in a table UI”
15. “Refactor React navigation into reusable sidebar components to reduce code duplication”
16. “Fix routing and authentication issues causing unexpected redirects to login page in React Router”
17. “Improve overall system integration between frontend and backend by aligning API contracts and data flow”
18. “check the endpoint error for chart forecast error that not populating in forecastApi.js”

All prompts were used for debugging assistance and architecture clarification only. Final implementation logic was customized and tested independently.